Cash, position and marked wealth

13 October 2026 | L001a accounting intuition | Synthetic fixed snapshots only

Sidiq review status: Not yet reviewed.

Research question and hypothesis

How do cash and a share position add up to wealth, and what changes when only the valuation price changes? Hypothesis: marked wealth equals cash plus the marked value of the shares. A changed mark changes that value but does not by itself put money into or take money out of the cash account.

Yesterday's C003a consolidation is integrated today with its true study date. Its repository evidence supports this bounded next step; it does not establish personal mastery. Today's sole new concept is valuation of an already owned position. We have not yet constructed a trade or an event-driven ledger.

Beginner explanation and terms

Cash C is money held in the account. Position q is the number of shares already owned. A mark p is an assumed valuation price per share. Marked holdings H are the money value assigned to that position. Wealth W is cash plus holdings in this example, which has no liabilities or other assets.

Think of two boxes: one contains money and one contains shares. Counting the money box alone misses the shares. Counting shares as if they were money mixes units. A mark translates each share into a money value for this snapshot. It does not promise that a buyer will pay that amount.

Investor.gov defines investment-company net asset value as assets less liabilities. We use that accounting idea in a deliberately narrower fixture: cash and one long position, zero liabilities. This is total account wealth in the toy model, not a fund's per-share NAV or an executable sale quote.

Mathematics, units and hand ledger

H = q * p. W = C + H = C + q * p. Units: q is shares; p is money units per share; H, C and W are money units. Multiplying shares by money units per share produces money units, so adding H to C is valid. Adding q directly to C would add unlike quantities. Marked wealth is not the same thing as cash.

Start with C=800 and q=4. At p=50, value each share at 50: H=50+50+50+50=200. W=800+200=1000. The position is supplied as an initial condition; we do not claim to have checked how it was acquired.

Snapshot	Cash (money)	Shares	Mark (money/share)	Holdings (money)	Wealth (money)
Initial	800	4	50	200	1000
Higher mark	800	4	55	220	1020
Lower mark	800	4	45	180	980

The higher and lower marks are separate alternatives to the initial snapshot. Neither is a trade. Relative to the initial mark, the four shares gain or lose 5 each, hence wealth changes by +20 or -20. Cash remains 800. This reconciles the entire change to the position; no cash receipt is invented.

Read the small C++ calculation slowly

The fixed example is cpp/projects/portfolio_accounting/marked_wealth.cpp. It reuses variables, integer arithmetic and output already seen in the savings project. No struct, vector or new parser is introduced. Each input is an explicit constant; the program accepts no external input.

1. #include <iostream> provides the output stream. int main() begins execution; its braces contain the statements. Reaching its end returns successful status from this simple demonstration.

2. const int cash = 800 and const int shares = 4 declare whole-number values that these statements cannot later change. cash is in money units; shares is a count. The type alone cannot enforce units.

3. initial_mark = 50, higher_mark = 55 and lower_mark = 45 are also const int values, measured in money units per share. These are chosen synthetic prices, not fetched observations or predictions.

4. initial_holdings = shares * initial_mark multiplies quantity by price. The next two declarations repeat that calculation for the alternative marks. Every intermediate in this fixture is at most 1020, safely within the minimum range required for int. No rounding is needed for these whole units.

5. Each std::cout statement prints its label, unchanged cash and share count, then its mark and holdings. The final cash + holdings expression adds two money amounts. The << operators send values to output; '\n' ends the line. Three statements print three snapshots; none updates an account.

Do not generalise this representation to arbitrary money inputs. Fractions, very large positions, negative states and overflow need an explicit contract before a reusable interface is introduced. These fixed nonnegative constants are the whole input domain of today's executable.

Protocol, implementation and checks

Predict the table before running. Build with make cpp-marked-wealth, which compiles with C++17 and -Wall -Wextra -Wpedantic -Werror, then runs scripts/verify_marked_wealth.py. The checker independently adds four Decimal prices, checks the hand-authored expected.txt, and compares exact stdout, empty stderr and exit 0 twice. Run without -O; an actual optimised invocation is verified to refuse execution.

The checker also reconciles +20 and -20 changes. Two complete checker invocations produce identical JSON, archived at experiments/portfolio_accounting/marked-wealth.json. This is deterministic fixture reproduction in the installed environment, not reproduction on a separately provisioned machine.

Observed: all three snapshots match on both runs. Apple Clang 21 compiles without diagnostics. A scratch C++ mutation replaces the three share-price multiplications with addition. It compiles, but the checker rejects its output with the intended mismatch and exit 1. Compilation success alone cannot establish that the arithmetic has the right units. Production source retains multiplication.

Full make check passes 608 Python tests, Ruff lint/54 formatted files, strict mypy/21 package files, 66 savings and 30 parser checks, 75 boundary comparisons, four prior golden outputs, fractional controls, eight readiness traces twice and today's three snapshots twice. Dependency consistency passes. Test counts are not coverage percentages; no new Python unit-test count is claimed.

Costs, failure cases and limitations

No trade occurs in these snapshots. Transaction fees, bid-ask spread, impact, tax, interest, dividends, settlement and liabilities are omitted; that omission is not an estimate of zero real costs. A liquidation at a different executable price or with fees could realise less cash than the marked holdings. Tomorrow's separate L001b lesson introduces one buy and an explicit fee.

A stale or unavailable mark can make this arithmetic uninformative even when the code is correct. A missing price must not silently become zero in a future data interface. Short positions, borrowing, fractional shares, multiple currencies and corporate actions are outside this fixture. None of the chosen marks is a historical observation. There is no historical period, fitting, signal or alpha test.

Looking only at cash misses the position. Adding share count to money mixes units. Treating the +20 mark change as a cash deposit double-counts wealth. The reference and compiled program still share our written assumptions; agreement does not validate those assumptions for a real account.

Primary sources consulted on October 13

Investor.gov, Net Asset Value: assets less liabilities supplies the accounting anchor; the single position and absence of liabilities are our fixture assumptions, not a complete NAV implementation. https://www.investor.gov/introduction-investing/investing-basics/glossary/net-asset-value

Microsoft, Built-in types: integral types represent whole numbers. The simple fixture uses small integer money units; real-money precision and range policy remain future design work. https://learn.microsoft.com/en-us/cpp/cpp/fundamental-types-cpp?view=msvc-170

Python, subprocess: captured output, return status and timeouts support exact executable checks. https://docs.python.org/3/library/subprocess.html#subprocess.run

Interview explanation

For one long position without liabilities, I reconcile wealth as cash plus quantity times its mark. Changing only the mark changes the position valuation, not cash. I check units and reconcile the wealth change independently. This is accounting validation, not evidence of a profitable strategy.

Progress, publication and necessary input

L001a is complete within its fixed-snapshot scope. Next: L001b, derive one buy and explicit fee. L001's reusable buy function and L002's delayed decisions remain unfinished. C003b, C004/C005 and Q004a/Q005b empirical gates are preserved. Sidiq's review remains unconfirmed; no answer is required for the next synthetic lesson. Licensed source evidence is still required before real-data work.

Today's persisted draw is 3. Two independent units are ready: yesterday's consolidation integration and today's valuation lesson. No third unit or extra lesson is manufactured. Publication uses actual commit dates, with remote verification reported separately. The daily schedule remains 08:00 Europe/London through December 31; stop new research thereafter and request the next scope.

Check your understanding

1. Why is cash=800 not the whole wealth at the initial mark?

2. With four shares, why does a rise from 50 to 55 add 20 to wealth but no cash?

3. Why is cash + shares an invalid wealth formula?

4. Does wealth=1020 guarantee that selling everything leaves 1020 cash?

Suggested answers

1. Four already owned shares are marked at 50 each, adding 200. Total marked wealth is 1000.

2. Each share is marked 5 higher: four additions of 5 total 20. There was no sale or other cash flow.

3. Cash is money and shares is a count. Multiply the count by a money-per-share price before adding.

4. No. The mark is a valuation assumption. Execution prices, fees and other omitted effects can change the realised cash; this lesson implements no sale or execution model.